

IMPLEMENT CLIENT SCRIPT & UI POLICY(INCIDENT)

PHASE 5

Project Development

5.1 Introduction

- The development phase focuses on implementing the required controls on the ServiceNow Incident table. UI Policies, UI Policy Actions, and Client Scripts are configured to provide conditional field control, automatic value handling, validation, and controlled user interaction.
- The implementation is divided into five milestones, with each milestone adding a specific functionality to the Incident form.

5.2 Development Process

The development process is carried out systematically using ServiceNow Incident Management features.

- ❖ Configure a UI Policy for High Impact Incidents.
- ❖ Configure a UI Policy Action to control the Urgency field.
- ❖ Implement an onChange Client Script for automatic Urgency updates.
- ❖ Implement an onSubmit Client Script for Incident validation.
- ❖ Implement an onCellEdit Client Script to control State changes from the Incident list.

Review each configuration and prepare it for functional testing.

5.3 Milestone 1: Create UI Policy on Incident

Activity 1: Create UI Policy

Step1: Log in to the ServiceNow instance with administrative or configuration access.

Step 2: Navigate to:

System UI → UI Policies

Step 3: Click New to create a new UI Policy record.

Step 4: Fill in the following details:

- Name: High Impact Control
- Table: Incident
- Active: true

Step 5: Configure the Conditions section as follows:

- Field: Impact
- Operator: is
- Value: 1 – High

Step 6: This condition ensures the UI Policy is triggered only when the Incident impact is set to High.

Step 7: Ui policy action Assignment group

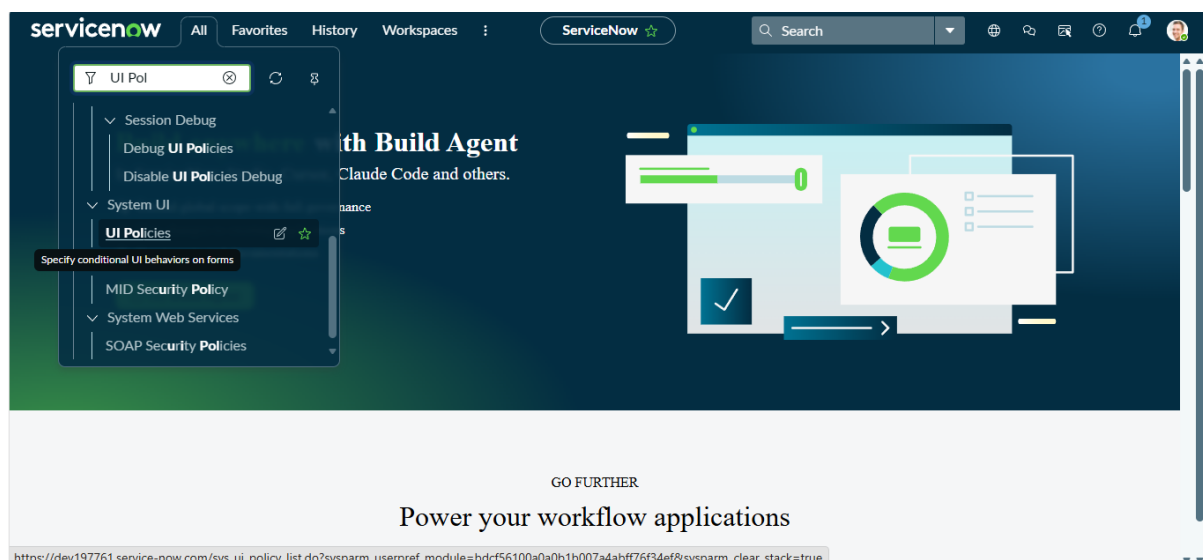
Step 8: Check mandatory

Step 9: Set Reverse if false to true.

Step 10: Review the configuration to ensure accuracy.

Step 11: Click Submit to save the UI Policy.

- Name: High Impact Control
- Table: Incident
- Active: true



The screenshot shows the 'UI Policy - High Impact Control' configuration page in ServiceNow. The page is titled 'UI Policy - High Impact Control' and has a 'View: Advanced' option. The configuration includes a 'Table' dropdown set to 'Incident [incident]' and an 'Application' dropdown set to 'Global'. The 'Active' checkbox is checked. The 'Short description' field contains 'High Impact Control' and the 'Order' field contains '100'. Below these fields are two tabs: 'When to Apply' and 'Script'. The 'When to Apply' tab is active and shows a 'Conditions' section with a dropdown set to 'Impact' and a value of '1 - High'. There are buttons for 'Add Filter Condition' and 'Add OR Clause'. Below the conditions are two blue boxes: 'If selected, the UI Policy applies to all form views; otherwise the UI Policy is view-specific' and 'Apply the UI policy actions when the form is loaded and when the user changes values on the form'. There are also checkboxes for 'Global' (checked), 'On load' (checked), 'Reverse if false' (checked), and 'Inherit' (unchecked).

The screenshot shows the 'UI Policy Action - assignment_group' configuration page in ServiceNow. The page is titled 'UI Policy Action - assignment_group' and has a 'More Info' link. The configuration includes a 'UI policy' dropdown set to 'High Impact Control' and a 'Table' dropdown set to 'Incident [incident]'. The 'Field name' dropdown is set to 'Assignment group'. The 'Application' dropdown is set to 'Global'. The 'Mandatory' dropdown is set to 'True', the 'Visible' dropdown is set to 'Leave alone', and the 'Read only' dropdown is set to 'False'. There is a 'Clear the field value' checkbox which is unchecked. At the bottom of the page are 'Update' and 'Delete' buttons.

5.4 Milestone 2: Create UI Policy Action – Urgency

Activity 1: Configure the Urgency Field

Step 1: Open the UI Policy Actions

- Go to the High Impact Control UI Policy you created earlier. Scroll down to the UI
- Policy Actions related list. This is where you define the specific behavior of fields when the policy conditions are met.

Step 2: Create a New Action

- Click New to create a UI Policy Action. This lets you configure how individual fields behave when the UI Policy is triggered.

Step 3: Fill in the Action Details

- Field name: Urgency
- Read-only: true
 - Visible: Leave this field as is (do not change).

Step 4: Submit the Action

- Click Submit to save the UI Policy Action. The action is now linked to the High Impact Control UI Policy and will automatically apply to the Urgency field when an Incident has High Impact.

Step 5: Optional Verification

To test the action:

- Open an Incident record.
- Set the Impact field to High.
- Check that the Urgency field becomes read-only.
- Change Impact to a different value to confirm that the field reverts back if needed.

The screenshot shows the ServiceNow interface for configuring a UI Policy Action. The breadcrumb trail is 'UI Policy Action - urgency'. A search bar is present. Below the header, there's a blue informational bar stating: 'UI policy actions specify exactly what actions to take on a specified field. The conditions specified in the UI policy determine when these actions are triggered. [More Info](#)'. The configuration form includes:

- UI policy:** High Impact Control
- Table:** Incident [incident]
- * Field name:** Urgency (dropdown menu)
- Application:** Global
- Mandatory:** Leave alone (dropdown menu)
- Visible:** Leave alone (dropdown menu)
- Read only:** True (dropdown menu)
- Clear the field value:** ☐

 At the bottom left, there are 'Update' and 'Delete' buttons. A small help icon is at the bottom right.

5.5 Milestone: Create onChange Client Script

Activity 1: Create Client Script

Step 1: Navigate to Client Scripts:

Go to System UI → Client Scripts.

Step 2: Click New to create a new Client Script.

Step 3: Fill in the details:

- Name: Auto set urgency for high impact – clearly shows the Purpose.
- Table: Incident – the script runs only on Incident records.
- Type: onChange – executes when a field value changes.
- Field name: Impact – triggers the script when Impact is Updated.
- Active: true – ensures the script runs immediately.

Step 4: Paste the script in the script editor section.

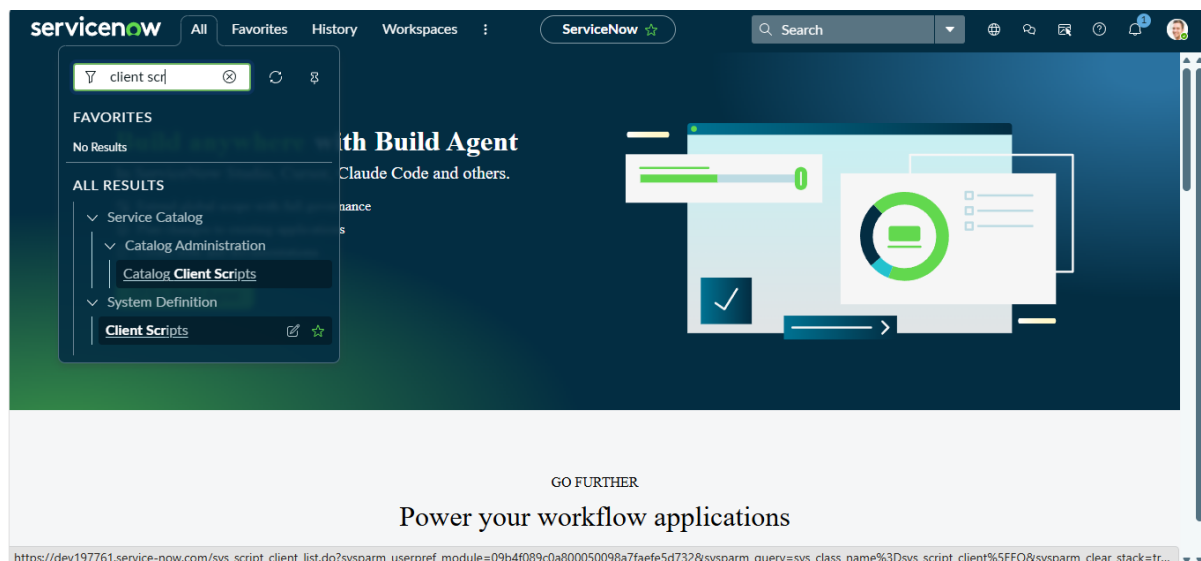
Step 5: Click Submit to save the Client Script.

Step 6: Verify: Open an Incident record and change the Impact field to High. The Urgency field should update automatically

Script:

```
function onChange(control, oldValue, newValue, isLoading) {  
    if (isLoading || newValue == '') {  
        return;  
    }  
    if (newValue == '1') {  
        g_form.setValue('urgency', '1');  
        g_form.addInfoMessage('Urgency set to High for High impact incident.');    }  
}
```

Step 7: Click Submit



servicenow All Favorites History Workspaces Admin Client Scripts						
Client Scripts Updated Search Actions on selected rows... New						
All > Class = Client Script						
☐ Q Name	Active	Table	Application	View	Type	Updated
Prevent state change via list edit	true	Incident [incident]	Global		onCellEdit	2026-09-23 12:06:41
Prevent state change via list edit	true	Incident [incident]	Global		onCellEdit	2026-09-23 12:03:00
Prevent save if Assigned To missing	true	Incident [incident]	Global		onSubmit	2026-09-23 09:54:58
Prevent save if Assigned To missing	true	Incident [incident]	Global		onSubmit	2026-09-23 09:47:49
Auto set urgency for high impact	true	Incident [incident]	Global		onChange	2026-09-23 09:20:28
Auto set urgency for high impact	true	Incident [incident]	Global		onChange	2026-09-23 09:14:18
Business Services Related List	false	Taxonomy Node [spn_taxonomy_node]	Global		onLoad	2026-06-12 14:24:45
MC - Make fields mandatory	true	Asset [alm_asset]	Global		onChange	2026-06-12 14:17:56
EF - Show Error Prefix Format Info	true	Error Source Application [sys_error_app]	Global		onLoad	2026-06-12 14:04:01
EF - Show Error Code Format Info	true	Error Code [sys_error_code]	Global		onLoad	2026-06-12 14:04:01
EF - AutoPopulateTargetDocumentTableField	true	Error Action [sys_error_action]	Global		onChange	2026-06-12 14:04:01
EF - Enricher Validate No IIFE	true	Error Enricher [sys_error_enricher]	Global		onSubmit	2026-06-12 14:04:01
EF - Auto-populate Severity Label	true	Error Code [sys_error_code]	Global		onChange	2026-06-12 14:04:01
EF - Clear M2M Depends On Source App	true	Error Action Mapping [sys_error_code_to_action_m2m]	Global		onChange	2026-06-12 14:04:01

servicenow
All Favorites History Workspaces Admin
Client Script - Auto set urgency for high impact
Search

Client Script
Auto set urgency for high impact
Update Delete

New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client.globals" to false.

Name
Auto set urgency for high impact
Table
Incident [incident]
UI Type
Desktop
Type
onChange
Field name
Impact

Application
Global
Active
Inherited
Global

Description

Messages

Script

```

1 function onChange(control, oldValue, newValue, isLoading) {
2   if (isLoading || newValue == '') {
3     return;
4   }
5   if (newValue == '1') {
6     g_form.setValue('urgency', '1');
7     g_form.addInfoMessage('urgency set to High for High Impact Incident.');
```

Isolate script

Update Delete

5.6 Milestone 4: Create onSubmit Client Script (Save Validation)

Activity 1: Create Client Script

Step 1: Navigate to Client Scripts:

Go to System UI → Client Scripts in your ServiceNow instance.

Step 2: Click New to create a new Client Script.

Step 3: Fill in the details:

- Name: Prevent save if Assigned To missing – clearly indicates that the script prevents saving if the field is empty.
- Table: Incident – applies the script only to Incident records.
- Type: onSubmit – executes when the user tries to save the record.
- Active: true – ensures the script is enforced immediately.

Step 4: Paste the script into the script editor section. This script should check if the Assigned To field is empty and prevent the record from saving if it is.

Step 5: Click Submit to save the Client Script.

Step 6: Verify: Try saving an Incident without filling the Assigned To field. The script should

prevent the save and display a warning.

script:

```
function onSubmit() {  
    if (g_form.getValue('impact') == '1' &&  
        g_form.getValue('assigned_to') == '') {  
        g_form.showErrorBox(  
            'assigned_to',  
            'Assigned To is mandatory for High impact incidents.'  
        );  
        return false;  
    }  
    return true;  
}
```

Step 7: Click Submit

The screenshot shows the ServiceNow interface for configuring a Client Script. The title bar indicates 'Client Script - Prevent save if Assigned To missing'. A blue notification banner at the top states: 'New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client.globals" to false.' The configuration form includes the following fields:

- Name:** Prevent save if Assigned To missing
- Table:** Incident [incident]
- UI Type:** Desktop
- Type:** onSubmit
- Application:** Global
- Active:** ☒
- Inherited:** ☐
- Global:** ☒
- Description:** (empty text area)
- Messages:** (empty text area)
- Script:**

```
1 function onSubmit() {  
2   if (g_form.getValue('impact') == '3') {  
3     g_form.setValue('assigned_to', '');  
4     g_form.showErrorBox(  
5       'assigned_to',  
6       'Assigned To is mandatory for High Impact Incidents.'  
7     );  
8     return false;  
9   }  
10  return true;  
11 }
```
- Isolate script:** ☒

At the bottom, there are 'Update' and 'Delete' buttons, and a 'Related Links' section with a link to 'Run Point Scan'.

5.7 Milestone 5: Create onCellEdit Client Script

Activity 1: Create Client Script

Step 1: Navigate to Client Scripts:

Go to System UI → Client Scripts in your ServiceNow instance.

Step 2: Click New to create a new Client Script.

Step 3: Fill in the details:

- Name: Prevent state change via list edit – clearly indicates that the script prevents users from changing the State field directly in a list.
- Table: Incident – applies the script only to Incident records.
- Type: onCellEdit – runs whenever a field is edited directly in a list view.
- Field name: State – the script will monitor changes to the State field.
- Active: true – ensures the script is enforced immediately.

Step 4: Paste the script into the script editor section. The script should prevent unauthorized changes to the State field when editing from the list view.

Step 5: Click Submit to save the Client Script.

Step 6: Verify: Open an Incident list and try to change the State field directly in the list. The script should block the change as intended.

Script:

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    alert ('State cannot be updated using list editing. Please open the Incident.');
```

```
    callback(false);
}
```

Step 7: Click Submit

The screenshot shows the ServiceNow interface for configuring a Client Script. The title bar indicates 'Client Script - Prevent state change via list edit'. A blue banner at the top provides information about strict mode and DOM access. The configuration form includes the following fields:

- Name:** Prevent state change via list edit
- Table:** Incident [incident]
- UI Type:** Desktop
- Type:** onCellEdit
- Field name:** State
- Application:** Global
- Active:** ☒
- Inherited:** ☐
- Global:** ☒
- Description:** (empty text area)
- Messages:** (empty text area)
- Script:**

```
1 function onCellEdit(sysIDs, table, oldValues, newValue, callback) { alert('State cannot be updated using list editing. Please open the Incident.');
```
- Isolate script:** ☒

At the bottom, there are 'Update' and 'Delete' buttons, and a 'Related links' section.

5.8 Conclusion :

The Project Development phase successfully translated the planned requirements and design into a functional ServiceNow Incident configuration. The required UI Policy, UI Policy Action, and Client Scripts were implemented to provide conditional field control, automatic field updates, input validation, and controlled record modification. The completed implementation establishes the core functionality of the project and provides a strong foundation for the Project Testing phase, where each implemented feature will be systematically verified.